Supplementary file to the paper

Secret Protection in Bounded Time Petri Nets Under the Indicator Semantics

(Paper ID: 26-0799)

Yifan Dong, Jakub Večeřa, Tomáš Masopust, and Zhiwu Li

This supplementary material provides additional technical and numerical results for the paper *Secret Protection in Bounded Time Petri Nets Under the Indicator Semantics*. Section 1 provides a complete computational complexity analysis of the proposed solution to the secret protection problem (SPP), accounting for the construction of the complete validity-aware modified state class graph (V-MSCG), candidate-policy evaluation, and the solution of the mixed-integer linear programs (MILPs) associated with the finite backward paths. To formally justify the state-class construction, Section 2 proves the soundness and completeness of Algorithm 1 with respect to the adopted time Petri net (TPN) semantics. The scalability of the complete procedure is then assessed in Section 3 through a parameterized case study that reports the benchmark model, implementation details, state-space statistics, and runtime results. Unless otherwise stated, all notation, algorithms, and numbered conditions follow those of the main paper.

1 Computational Complexity

This section analyzes the computational complexity of the complete procedure for solving the SPP. Let $\mathcal{G} = (Z, E, g, z_0)$ be the complete V-MSCG of a TPN $G = (N, M_0, I)$, and let T_p be the set of protectable transitions. The analysis proceeds in two stages. First, we evaluate the computational cost of constructing the complete V-MSCG as a one-time offline operation. Since the optimal policy is not known in advance, up to $2^{|T_p|}$ candidate policies $T_{pl} \subseteq T_p$ may need to be examined in the worst case. Second, for each candidate policy, we analyze the costs of constructing its pruned V-MSCG, computing the backward paths associated with the reachable secret vertices, and solving the MILPs defined by constraints (b1)–(b7) to verify its time validity.

Let N_{ex} denote the total number of vertex examinations in Algorithm 2, including the re-examinations required by newly generated duplicate branches. Let C_{adm} denote the worst-case cost of one transition-admissibility test, and let C_{inf} denote the worst-case cost of one infimum computation for a finite validity-clock expression. For each vertex examination, at most $|T|$ transitions are considered. For each transition, the admissibility test requires C_{adm} operations, Algorithm 1 requires $\mathcal{O}(|T|)$ operations, and updating the validity clocks requires at most $\mathcal{O}(|T_p|(C_{\text{inf}} + 1))$ operations. Consequently, constructing the complete V-MSCG has worst-case complexity $\mathcal{O}(N_{\text{ex}}|T|(C_{\text{adm}} + |T| + |T_p|(C_{\text{inf}} + 1)))$.

For a fixed candidate policy $T_{pl} \subseteq T_p$, pruning the branches inconsistent with T_{pl} and identifying all vertices reachable from z_0 require examining the vertices and edges of the complete V-MSCG. This step has worst-case complexity $\mathcal{O}(|Z| + |E|)$ and produces the pruned graph $\mathcal{G}_p(T_{pl}) = (Z_p, E_p, g_p, z_0)$. Let $Z_S \subseteq Z_p$ denote the set of reachable secret vertices of $\mathcal{G}_p(T_{pl})$. For every $z_s \in Z_S$, the backward paths associated with z_s are required to be computed. By Lemma 1 of the main paper, the set $\mathcal{B}(z_s)$ of backward paths is finite. Let L_s denote the maximum length of the

backward paths in $\mathcal{B}(z_s)$, and let $d^-_{\max}(T_{pl})$ denote the maximum in-degree of a vertex in $\mathcal{G}_p(T_{pl})$. Then, $|\mathcal{B}(z_s)| \leq \sum_{k=0}^{L_s} (d^-_{\max}(T_{pl}))^k$.

Let $L_{\max}(T_{pl}) = \max_{z_s \in Z_S} L_s$, and let $C^{\max}_{\mathrm{MILP}}(T_{pl})$ denote the maximum computational cost of solving one of the MILPs associated with these backward paths. Since constructing the constraint system associated with a backward path requires at most $\mathcal{O}(L_{\max}(T_{pl}))$ operations, the worst-case computational cost of backward-path generation and time-validity verification for a fixed T_{pl} is bounded by

$$\mathcal{O}\left(|Z_S| \sum_{k=0}^{L_{\max}(T_{pl})} \left(d^-_{\max}(T_{pl})\right)^k \left(L_{\max}(T_{pl}) + C^{\max}_{\mathrm{MILP}}(T_{pl})\right) \right).$$

Combining the one-time construction cost of the complete V-MSCG with the costs of pruning and verifying all candidate policies gives the following overall worst-case complexity:

$$\mathcal{O}\Bigg(N_{\mathrm{ex}}|T| \left(C_{\mathrm{adm}} + |T| + |T_p|(C_{\mathrm{inf}} + 1)\right) + \sum_{T_{pl} \subseteq T_p} \left(|Z| + |E| + |Z_S| \sum_{k=0}^{L_{\max}(T_{pl})} \left(d^-_{\max}(T_{pl})\right)^k \left(L_{\max}(T_{pl}) + C^{\max}_{\mathrm{MILP}}(T_{pl})\right)\right) \Bigg).$$

The above complexity accounts for the one-time construction of the complete V-MSCG, the pruning performed for each candidate policy, the computation of the backward paths associated with the reachable secret vertices, and the corresponding MILP-based verification. The summation over $T_{pl} \subseteq T_p$ explicitly reflects the fact that, in the worst case, up to $2^{|T_p|}$ candidate policies may need to be examined. The effects of cycles are retained through the backward paths, since repeated traversals that affect the timing information relevant to a secret vertex are represented by fresh firing-delay variables. By Lemma 1, however, the backward-path set associated with every reachable secret vertex remains finite under Assumptions A1 and A2.

The worst-case complexity remains exponential since both the size of the V-MSCG and the number of candidate policies may grow substantially. In practical computations, however, Lemma 2 and Remark 1 of the main paper allow entire families of policies to be excluded. If a policy is not time-valid, none of its subsets is time-valid; if a policy is time-valid, any positive-cost strict superset cannot be optimal. Examining policies in nondecreasing order of cost can therefore substantially reduce the number of policy-specific pruning and verification operations, although it does not change the exponential worst-case bound.

We now analyze the growth in the number of V-MSCG vertices relative to the MSCG constructed under the framework of TPN in [1, 2]. Each V-MSCG vertex $z = (M, \Theta, \varrho, \nu)$ augments an underlying MSCG state class (M, Θ) with the protection-decision function ϱ and the validity-clock vector ν. Hence, one MSCG state class may correspond to several V-MSCG vertices. Define $\hat{Z}(M, \Theta) = \{(M, \Theta, \varrho, \nu) \in Z\}$. Since $\varrho : T_p \to \{\mathrm{U}, \mathrm{Y}, \mathrm{N}\}$, at most $3^{|T_p|}$ protection-decision configurations are possible.

Let $T_0 = \{t \in T \mid \inf I(t) = 0\}$ with $T_+ = T \setminus T_0$. Under Assumption A2 of the main paper, no executable sequence in T_0^* is repetitive. Since the TPN is bounded, there exists a uniform finite bound r_0 on the number of consecutive firings of transitions in T_0. For each $t \in T_p$, define $Q_t = \sum_{s \in T_+} \left(1 + \left\lfloor \frac{te_{out}(t)}{\inf I(s)} \right\rfloor\right)$ and $B_t = (r_0 + 1)Q_t + r_0$. As established in Proposition 1 of the main paper, if more than B_t transition firings occur after the most recent reset of $\nu(t)$ without another reset, the accumulated elapsed time has a lower bound strictly greater than $te_{out}(t)$. Hence, Algorithm 2 replaces $\nu(t)$ with $+\infty$. Consequently, whenever $\nu(t) \neq +\infty$, it contains at most B_t firing-delay variables.

Let $B = \max_{t \in T_p} B_t$. Thus, every finite validity-clock component depends on a firing history of length at most B. For a fixed recent edge history of length at most B, each component $\nu(t)$ has at most $B+1$ possible reset positions, together with the inactive state $+\infty$. Let E_m denote the edge set of the underlying MSCG. Since the number of recent MSCG edge histories of length at most B is bounded by $\sum_{k=0}^{B} |E_m|^k$, the number of V-MSCG vertices corresponding to one MSCG state class satisfies $|\hat{Z}(M,\Theta)| \leq 3^{|T_p|}(B+2)^{|T_p|} \sum_{k=0}^{B} |E_m|^k$. Consequently, if the underlying MSCG contains $|Z_m|$ state classes, then $|Z| \leq |Z_m|\, 3^{|T_p|}(B+2)^{|T_p|} \sum_{k=0}^{B} |E_m|^k$.

This is a conservative worst-case upper bound. In practice, many combinations of protection decisions, validity-clock configurations, and recent edge histories are infeasible or unreachable due to the firing semantics and timing constraints of the TPN. Since the underlying MSCG is finite, the above bound also makes explicit that the complete V-MSCG is finite.

2 Soundness and Completeness of Algorithm 1

Algorithm 1 uses the symbolic, path-dependent representation of the MSCG. For a state class (M,Θ), the lower and upper firing-time bounds in Θ are symbolic expressions involving the firing-delay variables generated along the path leading to (M,Θ). These bounds are therefore interpreted jointly rather than as independent numerical intervals. More precisely, when transition t_i fires from a state class (M_j,Θ_j), the variable Δ_j denotes the symbolic time elapsed before this firing. For every transition t that remains enabled after the firing of t_i, its lower and upper bounds are updated as $\max\{0, l_t - \Delta_j\}$ and $u_t - \Delta_j$, respectively. The same variable Δ_j is used in the updated bounds of all such transitions and is retained in the resulting symbolic constraints. Consequently, the temporal relationships among the enabled transitions are preserved. The following lemma formally establishes that this state-class evolution is consistent with the adopted TPN semantics.

Lemma S.1 Let $G = (N, M_0, I)$ be a TPN, and let $\pi_{\mathcal{G}} = z_0 \rightarrow^{e_1} z_1 \rightarrow^{e_2} \cdots \rightarrow^{e_r} z_r$ be a structural path whose state-class components are generated by successive applications of Algorithm 1. Let $z_i = (M_i, \Theta_i, \varrho_i, \nu_i)$, for $i = 0, 1, \ldots, r$, and let $e_i = (z_{i-1}, z_i)$ with $g(e_i) = (t_i, \Delta_{j_i}, \kappa_i)$, for $i = 1, 2, \ldots, r$, where Δ_{j_i} is the graph-level firing-delay variable associated with the source vertex of e_i. Along $\pi_{\mathcal{G}}$, let δ_i denote the fresh firing-delay variable associated with the ith traversal of edge e_i. The transition sequence $t_1 t_2 \cdots t_r$ can fire from M_0 under the adopted TPN semantics if and only if the firing-delay variables $\delta_1, \delta_2, \ldots, \delta_r$ admit a common assignment satisfying the firing constraints along the path. Specifically, for every $i = 1, 2, \ldots, r$, this assignment satisfies $\max\{0, l_{t_i}\} \leq \delta_i < \min\{u_s \mid s \in En(M_{i-1})\}$, where l_{t_i} and u_s denote, respectively, the lower bound of t_i and the upper bound of s in Θ_{i-1}, after the assigned values of $\delta_1, \delta_2, \ldots, \delta_{i-1}$ have been substituted into these symbolic bounds.

Proof: We prove the result by induction on the length r of the path.

For $r = 0$, the path consists only of the initial vertex z_0. The empty transition sequence can fire from M_0, and no firing-delay variable or firing constraint is involved. Hence, the statement holds.

Assume that the statement holds for every structural path of length k. Consider a structural path of length $k+1$, i.e., $z_0 \rightarrow^{e_1} z_1 \rightarrow^{e_2} \cdots \rightarrow^{e_k} z_k \rightarrow^{e_{k+1}} z_{k+1}$, where the graph-level firing-delay variable labeling e_{k+1} is $\Delta_{j_{k+1}}$, and let δ_{k+1} be its fresh firing-delay instance associated with the $(k+1)$th traversal along the path. We show that the statement also holds for this path.

($\Rightarrow$) Suppose that the transition sequence $t_1 t_2 \cdots t_k t_{k+1}$ can fire from M_0. Its prefix $t_1 t_2 \cdots t_k$ can therefore fire from M_0. By the induction hypothesis, assigning the actual firing delays of this executable prefix to $\delta_1, \delta_2, \ldots, \delta_k$ satisfies all firing constraints along the path prefix ending at z_k.

Assign the actual time elapsed at M_k before the firing of t_{k+1} to δ_{k+1}. Since t_{k+1} fires after this elapsed time, its lower firing-time bound in Θ_k must have been reached. Therefore, $\delta_{k+1} \geq \max\{0, l_{t_{k+1}}\}$. Moreover, under the adopted strong semantics, no transition enabled at M_k may

reach its upper firing-time bound before t_{k+1} fires. Hence, $\delta_{k+1} < u_s$ for all $s \in En(M_k)$. It follows that

$$\max\{0, l_{t_{k+1}}\} \leq \delta_{k+1} < \min\{u_s \mid s \in En(M_k)\},$$

which is exactly the firing constraint generated by Algorithm 1 for the edge e_{k+1} after the graph-level firing-delay variable $\Delta_{j_{k+1}}$ is instantiated by the fresh path variable δ_{k+1}.

The firing of t_{k+1} produces $M_{k+1} = M_k + C(\cdot, t_{k+1})$, which is the marking computed by Algorithm 1. For every newly enabled transition $t' \in New(M_k, t_{k+1})$, Algorithm 1 assigns the initial firing domain $\theta_{t'} \in I(t')$. For every transition t' that remains enabled after the firing, suppose that its firing domain in Θ_k is $l_{t'} \leq \theta_{t'} < u_{t'}$. After the time represented by δ_{k+1} has elapsed, its remaining firing domain becomes $\max\{0, l_{t'} - \delta_{k+1}\} \leq \theta_{t'} < u_{t'} - \delta_{k+1}$, which is precisely the update performed by Algorithm 1 after the graph-level variable $\Delta_{j_{k+1}}$ is instantiated by δ_{k+1}. When the actual elapsed time is assigned to δ_{k+1}, these bounds coincide with the remaining firing-time bounds obtained after the timed firing of t_{k+1}. Thus, the assignment for the path prefix can be extended to δ_{k+1}, and all firing constraints along the path of length $k+1$ are satisfied.

($\Leftarrow$) Suppose that $\delta_1, \delta_2, \ldots, \delta_k, \delta_{k+1}$ admit a common assignment satisfying all firing constraints along the path. Restricting this assignment to $\delta_1, \delta_2, \ldots, \delta_k$, the induction hypothesis implies that $t_1 t_2 \cdots t_k$ can fire from M_0, reaching a timing state represented by (M_k, Θ_k).

The value assigned to δ_{k+1} satisfies

$$\max\{0, l_{t_{k+1}}\} \leq \delta_{k+1} < \min\{u_s \mid s \in En(M_k)\}.$$

The left inequality ensures that the lower firing-time bound of t_{k+1} is reached. The right inequality ensures that no transition enabled at M_k reaches its upper firing-time bound during this elapsed time. Therefore, time can elapse by the assigned value of δ_{k+1}, and t_{k+1} is firable under the adopted strong semantics.

After t_{k+1} fires, Algorithm 1 produces M_{k+1}, initializes the firing domains of newly enabled transitions, and updates the bounds of the transitions that remain enabled. These operations coincide with the firing-domain evolution prescribed by the adopted TPN semantics. Consequently, the timed execution of $t_1 t_2 \cdots t_k$ can be extended by t_{k+1}, and the complete sequence $t_1 t_2 \cdots t_k t_{k+1}$ can fire from M_0. Note that the same firing-delay variable δ_{k+1} is used in the updated bounds of every transition that remains enabled after the firing of t_{k+1}. Thus, all these bounds are updated using the same elapsed time and cannot evolve according to independently selected delays.

Therefore, by induction, a transition sequence can fire from M_0 if and only if the firing-delay variables along its corresponding structural path admit a common assignment satisfying all firing constraints. Hence, Algorithm 1 neither omits executable transition sequences nor introduces spurious feasible executions, establishing its completeness and soundness with respect to the adopted TPN semantics.

3 Parameterized Case Study

This section evaluates the scalability of the complete solution procedure on a parameterized TPN model of a flexible manufacturing cell. The model contains workpiece processing, inspection, and rework, together with two alternative authorization routes for a controlled-release request. By varying the initial number of workpieces, the benchmark produces a family of TPN instances of increasing size while retaining the same net structure, firing intervals, and protection parameters.

The physical meanings of the places and transitions in Fig. 1 are summarized in Table 1.

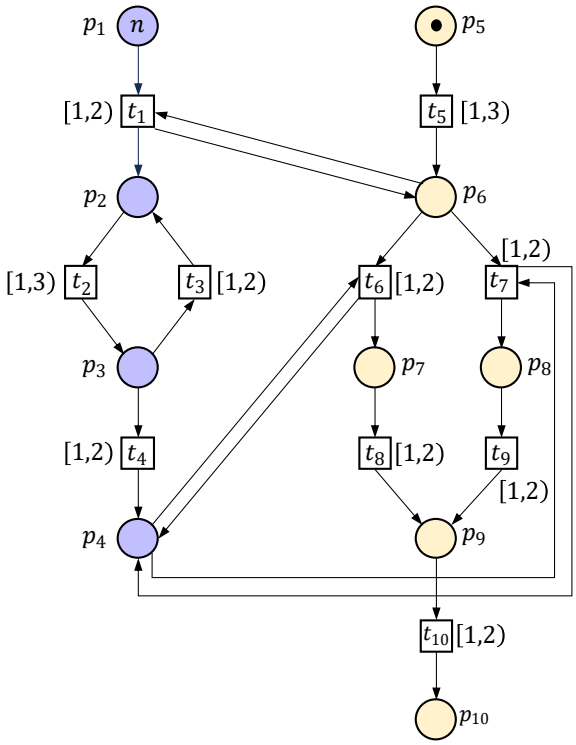


Figure 1: Parameterized TPN.

Table 1: Physical meanings of the places and transitions in the parameterized TPN.

Places	
p_1	Unprocessed workpieces waiting for loading
p_2	Processed workpieces waiting for inspection
p_3	Inspected workpieces awaiting acceptance or rework
p_4	Accepted workpieces satisfying the quality condition
p_5	Pending controlled-release request
p_6	Initialized request and processing-enabling condition
p_7	Request in authorization route A
p_8	Request in authorization route B
p_9	Authorized request waiting for final release
p_{10}	Completed release (indicator of a secret marking)
Transitions	
t_1	Load and process one workpiece
t_2	Inspect the processed workpiece
t_3	Return the workpiece for rework
t_4	Accept the inspected workpiece
t_5	Initialize the release-control procedure
t_6	Enter authorization route A
t_7	Enter authorization route B
t_8	Complete authorization route A
t_9	Complete authorization route B
t_{10}	Execute the final controlled release

The initial marking is $M_0 = np_1 + p_5$, where the n tokens in p_1 represent unprocessed workpieces and the token in p_5 represents a pending release request. Transition t_5 moves the request from p_5 to p_6. The self-loop between p_6 and t_1 supplies a nonconsuming synchronization condition for workpiece processing. Each workpiece is processed by t_1, inspected by t_2, and then either returned to p_2 by t_3 for rework or accepted by t_4 and moved to p_4.

Once p_4 is marked, the request may follow authorization route A, $t_6p_7t_8$, or authorization route B, $t_7p_8t_9$. Both routes merge at p_9, after which t_{10} moves the request to p_{10}. Therefore, every marking M_p satisfying $M_p(p_{10}) > 0$ is designated as secret.

The initialization and authorization operations t_5, t_6, and t_7 are protectable, and hence

$$T_p = \{t_5, t_6, t_7\}.$$

Their timeout values, clearance levels, and protection costs are fixed for all tested values of n:

$$\begin{aligned} te_{out}(t_5) = 6, \quad \gamma(t_5) = 1, \quad c(t_5) = 1, \\ te_{out}(t_6) = 4, \quad \gamma(t_6) = 1, \quad c(t_6) = 2, \\ te_{out}(t_7) = 4, \quad \gamma(t_7) = 1, \quad c(t_7) = 2. \end{aligned}$$

The required security level is $\ell(M_p) = 1$ for every secret marking $M_p \in S$. Since $|T_p| = 3$, at most $2^{|T_p|} = 8$ candidate policies are considered for each instance.

We vary the scale parameter from $n = 1$ to $n = 10$. For every n, the underlying MSCG and the complete V-MSCG $\mathcal{G} = (Z, E, g, z_0)$ are first constructed. The complete V-MSCG is constructed only once and reused for all candidate policies. For each $T_{pl} \subseteq T_p$, the branches inconsistent with T_{pl} are removed to obtain $\mathcal{G}_p(T_{pl}) = (Z_p, E_p, g_p, z_0)$. The backward paths associated with the reachable secret vertices are then computed, and the corresponding MILPs defined by constraints (b1)–(b7) are solved to determine whether T_{pl} is time-valid. Finally, the time-valid policy having the minimum total protection cost is returned as T_{pl}^*.

The complete procedure was implemented in MATLAB, and the MILPs associated with the backward paths were solved using `intlinprog` from the MATLAB Optimization Toolbox. All experiments were performed on a laptop equipped with an AMD Ryzen 5 230 processor and 32 GB of RAM.

Table 2 reports the following quantities: $|Z_m|$ and t_{MSCG}, the number of MSCG vertices and its construction time; $|Z|$ and $t_{\text{V-MSCG}}$, the number of complete V-MSCG vertices and its construction time; $\sum_{T_{pl} \subseteq T_p} |Z_p(T_{pl})|$, the total number of vertices in the policy-specific pruned graphs; t_{SPP}, the time used for policy pruning, backward-path computation, MILP-based verification, and optimal-policy selection; and $t_{\text{total}} = t_{\text{V-MSCG}} + t_{\text{SPP}}$. The last two columns give the optimal policy and its total cost.

Table 2: Scalability results for optimal secret protection policy synthesis.

n	$\lvert Z_m \rvert$	t_{MSCG} (s)	$\lvert Z \rvert$	$t_{\text{V-MSCG}}$ (s)	$\sum_{T_{pl} \subseteq T_p} \lvert Z_p(T_{pl}) \rvert$	t_{SPP} (s)	t_{total} (s)	T_{pl}^*	$\mathcal{C}^*$
1	9	0.02	57	0.02	152	0.00	0.02	$\{t_6, t_7\}$	4
2	104	0.04	2,433	1.40	5,164	0.03	1.43	$\{t_6, t_7\}$	4
3	344	0.18	10,983	9.41	22,804	0.23	9.64	$\{t_6, t_7\}$	4
4	943	0.59	17,061	16.56	35,976	0.35	16.91	$\{t_6, t_7\}$	4
5	1,824	1.32	25,595	24.16	55,192	0.38	24.53	$\{t_6, t_7\}$	4
6	2,984	2.21	36,302	33.93	79,260	0.46	34.39	$\{t_6, t_7\}$	4
7	4,421	3.36	48,927	44.98	106,136	0.58	45.56	$\{t_6, t_7\}$	4
8	6,135	4.84	63,815	56.96	137,600	0.76	57.73	$\{t_6, t_7\}$	4
9	8,126	6.46	81,059	71.37	174,024	0.95	72.31	$\{t_6, t_7\}$	4
10	10,394	8.35	100,659	87.27	215,408	1.22	88.49	$\{t_6, t_7\}$	4

As n increases from 1 to 10, the number of MSCG vertices increases from 9 to 10,394, while the number of complete V-MSCG vertices increases from 57 to 100,659. The additional growth results from the protection-decision function ϱ and validity-clock vector ν stored in every V-MSCG vertex. At $n = 10$, the V-MSCG contains approximately 9.7 times as many vertices as the underlying MSCG, and its construction time is approximately 10.5 times the MSCG construction time. Thus, construction of the complete V-MSCG accounts for most of the total computation time.

Nevertheless, the optimal SPP is solved within 88.49 s for the largest tested instance, which contains 100,659 V-MSCG vertices. The optimal policy is consistently $T_{pl}^{*} = \{t_6, t_7\}$ with $\mathcal{C}^{*} = 4$. These results quantify the additional cost of incorporating time-limited clearance and demonstrate that the proposed procedure can solve the tested systems of moderate size within a practical computation time.

References

[1] F. Basile, M. P. Cabasino, and C. Seatzu, "State estimation and fault diagnosis of labeled time Petri net systems with unobservable transitions," *IEEE Transactions on Automatic Control*, vol. 60, no. 4, pp. 997–1009, 2015.

[2] Z. He, Z. Li, A. Giua, F. Basile, and C. Seatzu, "Some remarks on 'State estimation and fault diagnosis of labeled time Petri net systems with unobservable transitions'," *IEEE Transactions on Automatic Control*, vol. 64, no. 12, pp. 5253–5259, 2019.